



Práctica 01: Relación Uno a Muchos

Objetivo

Implementar una relación **1:N** entre `Author` y `Post` usando SQLAlchemy 2.0.



Escenario

Crearás un modelo donde:

- Un **Author** puede tener muchos **Posts**
- Cada **Post** pertenece a un solo **Author**

Author (1) ———> Post (N)



Instrucciones

Paso 1: Configurar la Base de Datos

Abre `starter/database.py` y descomenta el código para configurar SQLAlchemy:

```
# Creamos el engine con SQLite
engine = create_engine("sqlite:///./blog.db", echo=True)

# SessionLocal para crear sesiones
SessionLocal = sessionmaker(bind=engine)

# Base para los modelos
Base = declarative_base()
```

El parámetro `echo=True` muestra las queries SQL en consola (útil para aprender).

Paso 2: Crear el Modelo Author

Abre `starter/models.py` y descomenta la clase `Author`:

```
class Author(Base):
    __tablename__ = "authors"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(100), unique=True)

    # Relación 1:N - Un autor tiene muchos posts
    posts: Mapped[list["Post"]] = relationship(back_populates="author")
```

Puntos clave:

- `Mapped[list["Post"]]` indica que es una lista de Posts
- `relationship()` crea la relación ORM
- `back_populates="author"` conecta con el otro lado

Paso 3: Crear el Modelo Post

Descomenta la clase `Post` en el mismo archivo:

```
class Post(Base):
    __tablename__ = "posts"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    content: Mapped[str] = mapped_column(Text)

    # Foreign Key - apunta al autor
    author_id: Mapped[int] = mapped_column(ForeignKey("authors.id"))

    # Relación inversa - cada post tiene un autor
    author: Mapped["Author"] = relationship(back_populates="posts")
```

Puntos clave:

- `ForeignKey("authors.id")` crea la FK en la tabla posts
- `Mapped["Author"]` (singular) indica un solo objeto
- `back_populates="posts"` conecta con la lista del Author

Paso 4: Crear las Tablas

Descomenta en `starter/main.py` :

```
from database import engine, Base
from models import Author, Post

# Crear todas las tablas
Base.metadata.create_all(bind=engine)
print("✅ Tablas creadas correctamente")
```

Ejecuta:

```
cd starter
python main.py
```

Deberías ver las queries CREATE TABLE en consola.

Paso 5: Insertar Datos

Descomenta la función `create_sample_data()` :

```
def create_sample_data():
    db = SessionLocal()

    # Crear autor
    author = Author(name="John Doe", email="john@example.com")
    db.add(author)
    db.commit()
    db.refresh(author)
    print(f"✅ Autor creado: {author.name} (ID: {author.id})")

    # Crear posts para el autor
```

```

post1 = Post(
    title="Intro to FastAPI",
    content="FastAPI es un framework moderno...",
    author_id=author.id # Usando FK directamente
)
post2 = Post(
    title="SQLAlchemy 2.0",
    content="SQLAlchemy 2.0 trae muchas mejoras...",
    author=author # Usando la relación (también válido)
)

db.add_all([post1, post2])
db.commit()
print(f"✅ Posts creados: {post1.title}, {post2.title}")

db.close()

```

Paso 6: Consultar Relaciones

Descomenta `query_relationships()` :

```

def query_relationships():
    db = SessionLocal()

    # Obtener autor con sus posts
    author = db.execute(
        select(Author).where(Author.email == "john@example.com")
    ).scalar_one()

    print(f"\n📝 Posts de {author.name}:")
    for post in author.posts: # Acceso via relación
        print(f"    - {post.title}")

    # Obtener post y su autor
    post = db.execute(select(Post).limit(1)).scalar()
    print(f"\n👤 Autor del post '{post.title}': {post.author.name}")

    db.close()

```

Paso 7: Ejecutar Todo

```
python main.py
```

Salida esperada:

```

✅ Tablas creadas correctamente
✅ Autor creado: John Doe (ID: 1)
✅ Posts creados: Intro to FastAPI, SQLAlchemy 2.0

📝 Posts de John Doe:
- Intro to FastAPI
- SQLAlchemy 2.0

```

 Autor del post 'Intro to FastAPI': John Doe

✓ Checklist de Verificación

- ☐ Las tablas se crean sin errores
 - ☐ Puedo crear un Author y múltiples Posts
 - ☐ Puedo acceder a `author.posts` (lista)
 - ☐ Puedo acceder a `post.author` (objeto)
 - ☐ Entiendo dónde va la ForeignKey (en el lado "muchos")
-

Reto Extra

Modifica el código para:

1. Crear un segundo autor con 3 posts
 2. Listar todos los autores con la cantidad de posts de cada uno
 3. Implementar `cascade="all, delete-orphan"` y probar eliminar un autor
-

[Siguiente: Práctica 02 - Relación N:M →](#)